RAG-Based Sales Data Analysis

Data Warehouse Course Hands-on Project

1. Project Overview

In this project, you will build a Retrieval-Augmented Generation (RAG) system to analyze a real-world sales dataset. You will use a vector database to store and retrieve relevant data, and integrate a Large Language Model (LLM) to answer analytical questions about sales trends, patterns, and insights.

This project bridges traditional data warehousing concepts with modern AI-powered analytics, demonstrating how LLMs can enhance data analysis workflows.

2. Team Requirements

This project can be completed individually or in pairs:

- Individual Project: 1 person per team

- Pair Project: Maximum 2 persons per team

Important: No team should have more than 2 members.

Requirements for 2-Person Teams:

If you work in a pair, your submission must include:

1. A clear division of work section in your technical report, specifying:

   - Which team member was responsible for each component

   - How the work was distributed (e.g., data preparation, vector database, RAG pipeline, testing)

   - Any collaborative tasks done together

2. Extended technical report: 2-3 pages (instead of 1-2 pages for individual projects)

3. Both team members must be present during the demo presentation

Send an email to the teacher (Jiaheng.lu@helsinki.fi) if you choose to build a 2-person team for this course by (31.03.2026)

Example Work Division:

Team Member	Responsibilities
Member A	Data preparation, chunking strategies, technical report
Member B	Vector database setup, RAG pipeline, demo preparation

3. Learning Objectives

By completing this project, you will:

1. Understand the architecture and workflow of RAG systems

2. Gain hands-on experience with vector databases and embeddings

3. Learn how to preprocess and chunk data for effective retrieval

4. Integrate open-source LLMs for data analysis tasks

5. Evaluate the effectiveness of RAG for analytical queries

4. Dataset

Use the Superstore Sales Dataset from Kaggle:

Download Link: https://www.kaggle.com/datasets/vivek468/superstore-dataset-final

Description: This dataset contains 4 years of sales data (2014-2017) from a retail superstore with 9,994 transactions

Key Fields:

- Order Date, Ship Date, Ship Mode

- Customer ID, Customer Name, Segment

- Product Category, Sub-Category, Product Name

- Sales, Quantity, Discount, Profit

- Region, State, City

Alternative Dataset Options:

- Online Retail Dataset: https://archive.ics.uci.edu/dataset/352/online+retail

- Brazilian E-Commerce Dataset: https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce

5. Technical Requirements

Component	Recommended Tools
Programming Language	Python 3.9+
Vector Database	ChromaDB, Milvus, or Qdrant
Embeddings	sentence-transformers (all-MiniLM-L6-v2) or HuggingFace
LLM	Ollama (Llama 3, Mistral) or HuggingFace models
RAG Framework	LangChain or LlamaIndex
Data Processing	Pandas, NumPy

6. Free LLM Options (No API Costs Required)

All LLMs used in this project are free and open-source. You do not need to pay for any API access.

Option 1: Ollama (Recommended)

Ollama lets you run LLMs locally with a simple command. Download from https://ollama.ai

Choose a model based on your hardware:

Model	Size	RAM Required	Command
Phi-3 Mini	2.2 GB	4 GB	ollama pull phi3
Gemma 2B	1.4 GB	4 GB	ollama pull gemma:2b
Llama 3.2 3B	2.0 GB	4 GB	ollama pull llama3.2:3b
Mistral 7B	4.1 GB	8 GB	ollama pull mistral
Llama 3 8B	4.7 GB	8 GB	ollama pull llama3

For students with limited hardware (4-8GB RAM, no GPU): Use Phi-3 Mini or Gemma 2B

Option 2: Google Colab (Free GPU Access)

If your laptop cannot run LLMs locally, use Google Colab free tier:

- Free access to T4 GPU (16GB VRAM)

- Can run larger models like Llama 3 8B or Mistral 7B

- Tutorial: https://colab.research.google.com/

Option 3: Free API Tiers

- Groq: Fast inference, generous free tier - https://console.groq.com

- Together AI: Free credits available - https://together.ai

7. System Architecture

Your system should implement the following pipeline:

Sales Dataset -> Data Processing and Chunking -> Text Chunks -> Vector Database

User Query -> Query Embedding -> Vector Search -> Retrieved Chunks -> LLM -> Response

8. Project Tasks

Part 1: Data Preparation (20%)

1. Load and explore the dataset

2. Create meaningful text representations of the data:

   - Convert each transaction to a natural language description

   - Create aggregated summaries (monthly sales, category performance, regional analysis)

   - Generate statistical summaries as text documents

3. Implement chunking strategies:

   - Experiment with different chunk sizes (500, 1000, 2000 characters)

   - Document your chunking approach and rationale

Part 2: Vector Database Setup (25%)

1. Set up a vector database (ChromaDB recommended for simplicity)

2. Generate embeddings for all text chunks

3. Store embeddings with appropriate metadata (date, category, region, etc.)

4. Implement retrieval functions with:

   - Similarity search

   - Metadata filtering

Part 3: RAG Pipeline Implementation (30%)

1. Set up a local LLM using Ollama or HuggingFace

2. Build the RAG pipeline:

   - Query processing

- Context retrieval (retrieve top-k relevant chunks)

- Prompt engineering for analytical responses

- Response generation

3. Implement conversation memory (optional but encouraged)

Part 4: Analysis Questions (25%)

Your system should be able to answer some of the following types of questions. Demonstrate at least 5 queries. You may create your own queries for the data sets.

Trend Analysis:

- What is the sales trend over the 4-year period?

- Which months show the highest sales? Is there seasonality?

- How has profit margin changed over time?

Category Analysis:

- Which product category generates the most revenue?

- What sub-categories have the highest profit margins?

- Which products are frequently sold at a discount?

Regional Analysis:

- Which region has the best sales performance?

- Compare sales performance across different states.

- Which cities are the top performers?

Comparative Analysis:

- Compare Technology vs. Furniture sales trends.

- How does the West region compare to the East in terms of profit?

9. Deliverables

9.1 Source Code

- Well-documented Python code

- requirements.txt or environment.yml

- README with setup instructions

- Submit via GitHub repository or zip file

9.2 Technical Report

Individual project: 1-2 pages

Team project (2 persons): 1-3 pages (must include work division section)

Report should include:

- System architecture description

- Data preprocessing and chunking approach

- Embedding model and vector database choices (with justification)

- LLM selection and prompt engineering strategies

- Sample queries and responses (with screenshots)

- Challenges encountered and solutions

- [For teams] Division of work between team members

9.3 Demo Presentation (7-8 minutes)

- System walkthrough

- Live demonstration of 5+ analytical queries

- Discussion of results accuracy

- [For teams] Both members must present

10. Evaluation Rubric

Criteria	Points
Data preparation and chunking quality	6
Vector database implementation	6
RAG pipeline functionality	6
Answer quality and accuracy	6
Code quality and documentation	6
Total	30

11. Resources

Documentation:

- LangChain: https://python.langchain.com/docs/

- ChromaDB: https://docs.trychroma.com/

- Ollama: https://ollama.ai/

- Sentence Transformers: https://www.sbert.net/

Tutorials:

- RAG from Scratch: https://github.com/langchain-ai/rag-from-scratch

- Building RAG Applications: https://www.deeplearning.ai/short-courses/

Book:

- LLM engineer's handbook : master the art of engineering large language models from concept to production / Paul Iusztin, Maxime Labonne. 2024

Available in the library of University of Helsinki

12. Important Dates

- LLM and RAG presentation and QA : March 20, 2026, 10:15 AM – 12:00 PM

- Vector database presentation and QA: April 17, 2026, 10:15 AM – 12:00 PM

- Final Submission: 30/4/2026

13. Academic Integrity

This project can be completed individually or in pairs (maximum 2 persons per team). You may discuss concepts with classmates, but all code and written work must be your own (or your teams own work). Properly cite any external resources, tutorials, or code snippets you reference.

14. AI Tool Usage Policy

Students are allowed to use AI tools (e.g., ChatGPT, Claude, GitHub Copilot) to help generate code for this project. However, simply submitting AI-generated code without explanation, problem analysis, and evidence of personal contribution is not acceptable. The goal is to learn from the process, not just produce output.

Requirements for AI Tool Usage:

1. Disclosure: Clearly describe in the technical report which AI tools were used and for which parts of the project (e.g., data preparation, vector database setup, RAG pipeline, analysis queries). Include the specific prompts or instructions given to the AI tool.

2. Problem Analysis: Explain any problems, errors, or limitations encountered in the AI-generated code — including bugs, incorrect outputs, performance issues, or cases where the code did not work as expected.

3. Running Results: Document the actual running results with screenshots or output logs. Show what worked, what failed, and how issues were resolved.

4. Student Contribution: Clearly state what the student(s) contributed beyond the AI-generated code — such as debugging, modifying, integrating, testing, prompt engineering, architectural decisions, and analysis interpretation. The report must demonstrate genuine understanding of the code and the RAG system.

The technical report must include a dedicated section on AI tool usage covering the above four points. This section does not count toward the page limit.

Good luck with your project!